

OpenShift v4: 25 Practical Day-to-Day Scenarios & Solutions

Get one to one assistance for OpenShift Hands on labs (50 labs).
WhatsApp Dhinesh +91 9444410227 and get started today!

<https://assistedcloud.com/>

1. Scenario: Check the status of all cluster nodes.

Solution: Use oc get nodes to list nodes and their status (Ready, SchedulingDisabled, etc.).
oc get nodes -o wide

2. Scenario: Temporarily stop scheduling new pods onto a specific node (e.g., for maintenance).

Solution: Cordon the node. Existing pods remain, but no new ones are scheduled.
oc adm cordon <node_name>

3. Scenario: Safely remove all application pods from a node before maintenance (like rebooting).

Solution: Drain the node. This cordons the node first and then evicts pods gracefully (respecting PodDisruptionBudgets).

Ensure you have appropriate PodDisruptionBudgets set for your apps first!

oc adm drain <node_name> --ignore-daemonsets --delete-emptydir-data

- ✓ --ignore-daemonsets: Needed as DaemonSet pods aren't evicted.
- ✓ --delete-emptydir-data: Acknowledges that data in emptyDir volumes will be lost.

4. Scenario: Allow scheduling back onto a node after maintenance.

Solution: Uncordon the node.
oc adm uncordon <node_name>

5. Scenario: Check the logs of a specific pod.

Solution: Use oc logs.
oc logs <pod_name> [-n <project_name>]
Follow logs in real-time
oc logs -f <pod_name> [-n <project_name>]
See logs for a previous instance of a pod (if it restarted)
oc logs --previous <pod_name> [-n <project_name>]

To get the OpenShift Free downloads, please visit: <https://assistedcloud.com/free-downloads/>

6. Scenario: A pod is stuck in CrashLoopBackOff. How to investigate?

Solution:

1. Check logs: `oc logs <pod_name>` for application errors.
2. Check previous logs: `oc logs --previous <pod_name>` if it ran briefly before crashing.
3. Describe the pod: `oc describe pod <pod_name>` - Look at Events, container exit codes, and reasons. Often indicates issues like OOMKilled, failed probes, or configuration errors.
4. Exec into the container (if possible): `oc exec <pod_name> -it -- /bin/bash` to explore the environment.

7. Scenario: Check resource usage (CPU/Memory) for pods or nodes.

Solution: Use `oc adm top`. Requires metrics server to be running (usually is by default).

Top nodes

`oc adm top nodes`

Top pods in the current project

`oc adm top pods`

Top pods in a specific project

`oc adm top pods -n <project_name>`

8. Scenario: Scale a deployment up or down.

Solution: Use `oc scale`.

`oc scale deployment/<deployment_name> --replicas=5 [-n <project_name>]`

Works for DeploymentConfig (dc) as well

`oc scale dc/<deploymentconfig_name> --replicas=3 [-n <project_name>]`

9. Scenario: Trigger a new rollout/deployment for an application.

Solution: Use `oc rollout latest`.

`oc rollout latest deployment/<deployment_name> [-n <project_name>]`

Or for DeploymentConfig

`oc rollout latest dc/<deploymentconfig_name> [-n <project_name>]`

10. Scenario: Check the status and history of a deployment.

Solution: Use `oc rollout status` and `oc rollout history`.

Watch rollout progress

`oc rollout status deployment/<deployment_name> [-n <project_name>]`

View past revisions

`oc rollout history deployment/<deployment_name> [-n <project_name>]`

View details of a specific revision

`oc rollout history deployment/<deployment_name> --revision=3 [-n <project_name>]`

11. Scenario: Roll back a deployment to a previous version.

Solution: Use oc rollout undo.

Roll back to the immediately previous revision

```
oc rollout undo deployment/<deployment_name> [-n <project_name>]
```

Roll back to a specific revision number (from history)

```
oc rollout undo deployment/<deployment_name> --to-revision=2 [-n <project_name>]
```

12. Scenario: Expose a deployment externally using a Route.

Solution: Use oc expose.

Expose a service associated with a deployment (assuming service exists)

```
oc expose service/<service_name> [--port=<service_port>] [--name=<route_name>] [-n <project_name>]
```

Example: Expose service 'my-app-svc' port 8080

```
oc expose service/my-app-svc --port=8080 --name=my-app-route
```

13. Scenario: Get the external URL for an application.

Solution: Use oc get route.

```
oc get route <route_name> -o jsonpath='{.spec.host}' [-n <project_name>]
```

List all routes in the project

```
oc get routes [-n <project_name>]
```

14. Scenario: A pod can't connect to another service within the cluster.

Solution:

1. Verify Service Existence: `oc get svc <service_name> [-n <service_project>]`. Check the ClusterIP and Port.
2. Verify Service Endpoints: `oc get endpoints <service_name> [-n <service_project>]`. Ensure there are healthy pod IPs listed. If not, check the target pods' readiness probes and labels match the service selector.
3. Check Network Policies: `oc get networkpolicy [-n <project_name>]`. Ensure no policy is blocking traffic from the source pod/project to the destination service/project.
4. DNS Check: Exec into the source pod (`oc exec <pod_name> -it -- /bin/sh`) and try `curl <service_name>.<project_name>.svc.cluster.local:<port>`. Check `/etc/resolv.conf`.

15. Scenario: Add a label to a node (e.g., to target specific workloads).

Solution: Use oc label node.

```
oc label node <node_name> region=east
```

Add a label to target GPU workloads

```
oc label node <node_name> accelerator=nvidia-tesla-v100
```

You can then use nodeSelector or nodeAffinity in pod specs.

16. Scenario: Check Persistent Volume Claim (PVC) status and bound Persistent Volume (PV).

Solution: Use oc get pvc.

```
oc get pvc [-n <project_name>]
```

Describe a specific PVC to see the bound PV name and capacity

```
oc describe pvc <pvc_name> [-n <project_name>]
```

17. Scenario: Check available Storage Classes.

Solution: Use oc get storageclass.

```
oc get sc # sc is short for storageclass
```

18. Scenario: Get a temporary API token for a service account.

Solution: Use oc create token (OCP 4.6+). For older versions, you might need to extract it from the service account's secret.

OCP 4.6+

```
oc create token <service_account_name> [-n <project_name>] [--duration=1h]
```

Older versions (less secure, token doesn't expire automatically)

Find the token secret associated with the SA

```
# oc get sa <service_account_name> -o yaml
```

```
# oc get secret <token_secret_name> -o jsonpath='{.data.token}' | base64 --decode
```

19. Scenario: Grant a user permissions to view resources in a specific project.

Solution: Add the view role to the user for that project.

```
oc policy add-role-to-user view <user_name> -n <project_name>
```

Other common roles: edit, admin.

20. Scenario: Check the status of Cluster Operators.

Solution: Use oc get clusteroperator. Healthy operators are crucial for cluster stability.

```
oc get clusteroperator
```

Describe a specific operator if it's degraded or unavailable

```
oc describe clusteroperator <operator_name>
```

21. Scenario: Find out which pods are using the most memory in a project.

Solution: Use oc adm top pods and sort by memory.

```
oc adm top pods -n <project_name> --sort-by=memory
```

22. Scenario: A pod fails to start with CreateContainerConfigError.

Solution: Describe the pod: `oc describe pod <pod_name>`. Look at the Events section. This error often means OpenShift couldn't find a referenced ConfigMap or Secret needed to configure the container (e.g., referenced in `envFrom` or `volumeMounts`). Verify the ConfigMap/Secret exists in the same namespace and is correctly named.

23. Scenario: Check project resource quotas and limits.

Solution: Use `oc describe quota` and `oc describe limitrange`.
 # Check resource quotas (limits on total resources in project)
`oc describe resourcequota <quota_name> -n <project_name>`
 # Or list all
`oc get resourcequota -n <project_name>`

 # Check limit ranges (default requests/limits for containers)
`oc describe limitrange <limitrange_name> -n <project_name>`
 # Or list all
`oc get limitrange -n <project_name>`

24. Scenario: Copy files to/from a running container.

Solution: Use `oc cp`.
 # Copy local file to pod
`oc cp ./myfile.txt <pod_name>:/tmp/myfile.txt [-n <project_name>]`

 # Copy file from pod to local
`oc cp <pod_name>:/app/logs/server.log ./server.log [-n <project_name>]`

25. Scenario: A pod is failing due to Security Context Constraints (SCC) issues (e.g., unable to validate against any security context constraint).

Solution:

1. Describe the pod: `oc describe pod <pod_name>` - check Events.
2. Identify Pod's Service Account: `oc get pod <pod_name> -o jsonpath='{.spec.serviceAccountName}'`.
3. Check SCCs allowed for the Service Account: `oc adm policy who-can use scc <scc_name>` (try common ones like `restricted`, `anyuid`, `nonroot`). Or check the SA's roles/groups to see which SCCs they map to.
4. Analyze Pod Security Context: Check the `securityContext` defined in the pod/container spec (e.g., `runAsUser`, `fsGroup`, `allowPrivilegeEscalation`, `capabilities`).

5. Compare: Ensure the pod's requirements are permitted by at least one of the SCCs available to its service account. You might need to:
 - Adjust the pod's securityContext to comply with an allowed SCC (e.g., remove privileged settings).
 - Grant the service account access to a more permissive SCC (use `oc adm policy add-scc-to-user <scc_name> -z <service_account_name>`), but do this cautiously.
 - Create a custom SCC if necessary.

This list provides a starting point for tackling common OpenShift v4 tasks. Remember to replace placeholders like `<node_name>`, `<pod_name>`, `<project_name>`, etc., with your actual resource names. Always consult the official OpenShift documentation for detailed explanations and advanced options.

**Get one to one assistance for OpenShift Hands on labs (50 labs).
WhatsApp Dhinesh +91 9444410227 and get started today**

To get the OpenShift Free downloads, please visit: <https://assistedcloud.com/free-downloads/>